

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Interaktivní nástroj pro kryptografické algoritmy



2026

Vedoucí práce:
doc. RNDr. Eduard Bartl, Ph.D.

Jaroslav Vymětal

Studijní program: Informační technologie,
prezenční forma

Bibliografické údaje

Autor: Jaroslav Vymětal
Název práce: Interaktivní nástroj pro kryptografické algoritmy
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informační technologie, prezenční forma
Vedoucí práce: doc. RNDr. Eduard Bartl, Ph.D.
Počet stran: 37
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Jaroslav Vymětal
Title: Interactive tool for cryptographic algorithms
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Information Technologies, full-time form
Supervisor: doc. RNDr. Eduard Bartl, Ph.D.
Page count: 37
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Výuka kryptografie na akademické půdě vychází z pevných matematických základů. Přestože je tento formální přístup pro hluboké porozumění oboru nezbytný, v počátečních fázích studia může pro mnohé představovat bariéru. Tato bakalářská práce představuje interaktivní webovou aplikaci, která prostřednictvím přehledné vizualizace usnadňuje studentům pochopení daných kryptografických mechanismů. V práci shrnuji principy vybraných symetrických a asymetrických šifer a popisuji samotný vývoj aplikace.

Synopsis

The teaching of cryptography in an academic environment is based on solid mathematical foundations. Although this formal approach is essential for a deep understanding of the field, it can present a barrier for many in the early stages of their studies. This bachelor thesis presents an interactive web application that facilitates students' understanding of the given cryptographic mechanisms through clear visualization. In the thesis, I summarize the principles of selected symmetric and asymmetric ciphers and describe the actual development of the application.

Klíčová slova: kryptografie, vizualizace, šifrovací algoritmy, webová aplikace, vzdělávání

Keywords: cryptography, visualization, encryption algorithms, web application, education

Děkuji vedoucímu práce doc. RNDr. Eduardu Bartlovi, Ph.D. za odborné vedení a cenné rady při tvorbě této práce.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	7
1.1	Motivace	7
1.2	Cíl práce	7
2	Teoretický rozbor kryptografie	8
2.1	Úvod do kryptografie	8
2.2	Symetrické šifry	9
2.2.1	Klasické šifry	9
2.2.2	Proudové šifry	10
2.2.3	Blokové šifry	11
2.3	Asymetrické šifry	13
2.3.1	Kryptosystém RSA	14
2.3.2	Bezpečnost RSA	14
2.3.3	Matematický princip dešifrování RSA	15
3	Použité technologie	16
3.1	Frontendové technologie a standardy	16
3.2	Vue.js	17
3.3	Vývojové prostředí a sestavení aplikace	17
4	Návrh a implementace aplikace	19
4.1	Struktura projektu	19
4.2	Logika šifer	20
4.3	Vizualizace	26
4.3.1	Návrh vizualizace	26
4.3.2	Implementace vizualizace pomocí Vue.js	26
4.4	Logo	27
5	Prezentace dosažených výsledků	28
5.1	Ukázky průchodu aplikací	28
5.2	Požadavky na prohlížeč a zařízení	33
5.3	Rozšíření o další šifry	33
	Závěr	34
	Conclusions	35
A	Obsah elektronické přílohy	36
	Literatura	37

Seznam obrázků

1	Struktura projektu aplikace	20
2	Logo aplikace	27
3	Domovská stránka aplikace	29
4	Rozhraní posuvné šifry	30
5	Rozhraní proudové šifry s LFSR	31
6	Rozhraní pro kryptosystém RSA	32

Seznam zdrojových kódů

1	Korekce záporného modula	21
2	Výpočet modulární inverze	21
3	Šifrování pomocí mapování indexů	22
4	Řízení indexu klíče při ignorování mezer a teček	23
5	Výpočet zpětné vazby LFSR	23
6	Průběh šifrovací rundy DES	24
7	Modulární umocňování s využitím BigInt	25
8	Ukázka použití direktiv u posuvníku klíče	27

1 Úvod

Bezpečná komunikace a ochrana dat jsou v dnešní digitální době nezbytnou součástí informačních technologií. Šifrování využíváme denně, ať už při prohlížení internetu, placení kartou nebo posílání zpráv. Málokoho z běžných uživatelů napadne pozastavit se nad tím, co všechno se v pozadí odehrává. Budoucí informatici a specialisté na kybernetickou bezpečnost by však měli těmto procesům rozumět. Studium kryptografie ovšem nemusí být v počátečních fázích pro studenty snadné. Tato disciplína totiž vyžaduje pochopení řady netriviálních matematických principů a abstraktních konceptů. Jelikož se mnohdy vyučuje primárně pomocí vzorců a formálních definic, chybí studentům možnost okamžité vizuální zpětné vazby, která by názorně ukázala, co se se šifrovanými daty reálně děje. Právě toto množství teoretických informací se pro začínající studenty může stát bariérou. Tato práce proto vznikla s cílem nabídnout interaktivní webovou aplikaci, která pochopení kryptografických principů usnadní pomocí přehledné vizualizace.

1.1 Motivace

Kryptografie má sice bohatou historii sahající několik staletí před náš letopočet [12], avšak dnešní šifrovací standardy stojí na pevných matematických základech. Tento formální přístup je pro obor nezbytný, ale při prvním kontaktu dokáže působit demotivačně. Běžné vysokoškolské učebnice často definují šifry pomocí strohých rovnic. Student si pak musí při čtení neustále v mysli převádět symboly na reálné transformace dat, což ztěžuje pochopení samotného principu.

S tímto problémem jsem se během svého studia rovněž potýkal. Snaha pochopit fungování algoritmu čistě z matematického zápisu často vedla k mechanickému učení bez hlubšího chápání souvislostí. Chyběl mi nástroj, který by složitou teorii převedl do intuitivnější vizuální podoby, a právě to mě motivovalo k vytvoření vlastní aplikace.

1.2 Cíl práce

Záměrem je poskytnout studentům prostředí, kde si mohou principy vybraných šifer sami interaktivně vyzkoušet. U každé šifry nabízí aplikace vizuální znázornění, které odpovídá tomu, co algoritmus skutečně dělá, čímž eliminuje nutnost provádět složité mentální operace. Základem navrženého rozhraní je okamžitá zpětná vazba. Uživatel může plynule měnit vstupní parametry, jako je text či klíč, a v reálném čase sledovat, jak se mění výsledný šifrový text. Nástroj nemá za úkol nahradit formální matematický výklad, ale spíše sloužit jako přechodový můstek. Student si pomocí vizualizace a experimentování nejprve vybuduje intuitivní porozumění problému, na které se následně mnohem snáze navazuje přesnou definicí a teorií.

2 Teoretický rozbor kryptografie

V této kapitole vysvětluji principy, podle kterých jsem naprogramoval algoritmy v praktické části práce. Vybírám jen ty pojmy a matematická pravidla, která jsou nutná pro pochopení algoritmů v mé aplikaci. Popisuji zde, co je to kryptosystém, co znamenají pojmy důvěrnost a integrita dat a jaký je rozdíl mezi symetrickou a asymetrickou kryptografií. Vycházím přitom z knihy od Stinsona a Patersona [13].

2.1 Úvod do kryptografie

Kryptografie se zabývá metodami pro zabezpečení komunikace. Historicky se jednalo především o jednoduché substituční a transpoziční šifry, dnes však moderní kryptografie stojí na pevném matematickém základě a veřejně známých algoritmech. Historií kryptografie se v této práci podrobně nezabývám, přestože je bohatá a zajímavá, neboť není náplní této práce. Základní situace vypadá takto: Odesílatel (Alice) chce poslat zprávu příjemci (Bobovi) přes nezabezpečený kanál. Musí to udělat tak, aby případný útočník (Oskar) zprávu nepřečetl ani nezměnil.

Formální definice kryptosystému

Abych mohl algoritmy v aplikaci správně implementovat, musím si nejprve definovat, z čeho se takový šifrovací systém skládá. Podle Stinsona [13] je kryptosystém pětice $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$, kde

- $\mathcal{P}$ je množina otevřených textů (plaintext),
- $\mathcal{C}$ je množina zašifrovaných textů (ciphertext),
- $\mathcal{K}$ je množina klíčů (keys),
- $\mathcal{E}$ a $\mathcal{D}$ jsou množiny pravidel pro šifrování a dešifrování (encrypt, decrypt).

Aby systém fungoval, musí platit základní podmínka korektnosti kryptosystému: pokud se zpráva zašifruje a pak správným klíčem dešifruje, výsledkem musí být původní text:

$$d_K(e_K(x)) = x. \quad (1)$$

Základní bezpečnostní cíle a Kerckhoffsův princip

Kryptografie nejenže musí zajistit, že zpráva zůstane tajná, ale také že nebude možné ji změnit bez povšimnutí. Proto se často mluví o dvou hlavních bezpečnostních cílech, a to o důvěrnosti a integritě dat. Důvěrnost zajišťuje, že si zprávu nepřečte nikdo nepovolaný. Tuto vlastnost v aplikaci demonstrují přeměnou vstupního textu na nečitelnou změť znaků. Integrita dat naopak zajišťuje, že zprávu nikdo cestou nezměnil. I když moje aplikace vizualizuje hlavně šifrování (důvěrnost), integrita je pro bezpečnost stejně tak důležitá.

S těmito cíli úzce souvisí i Kerckhoffsův princip [7]. Podle něj by bezpečnost šifry neměla záviset na utajení algoritmu, ale pouze na utajení klíče. To znamená, že i když by útočník znal celý postup šifrování, neměl by být schopen zprávu dešifrovat bez znalosti správného klíče. Tento princip je základem pro moderní kryptografii a zajišťuje, že algoritmy mohou být veřejně analyzovány a testovány na bezpečnost.

2.2 Symetrické šifry

Symetrická kryptografie, v literatuře často označovaná jako kryptografie s tajným klíčem, představuje historicky nejrozšířenější typ šifrování. Základním principem je, že odesílatel i příjemce sdílejí tentýž tajný klíč K . Tento klíč je použit jak pro šifrovací funkci e_K , tak pro dešifrovací funkci d_K . Bezpečnost celého systému závisí na tom, že se klíč K nedostane do rukou útočníka.

2.2.1 Klasické šifry

Do této kategorie patří historické algoritmy, které byly vyvíjeny před nástupem digitálních počítačů. Pracuje se zde s anglickou abecedou o 26 znacích, kterou pro matematické účely mapují na množinu celých čísel $\mathbb{Z}_{26} = \{0, 1, \dots, 25\}$. Ačkoliv jsou tyto šifry z dnešního pohledu snadno prolomitelné, ilustrují základní principy substituce.

Posuvná šifra

Jedná se o nejjednodušší formu substituční šifry. Klíčem je číslo $K \in \mathbb{Z}_{26}$. Šifrování a dešifrování znaku x probíhá pomocí modulární aritmetiky:

$$e_K(x) = (x + K) \bmod 26, \quad (2)$$

$$d_K(y) = (y - K) \bmod 26. \quad (3)$$

Nejznámějším případem je *Caesarova šifra*, kde je klíč pevně stanoven na hodnotu $K = 3$.

Afinní šifra

Tato šifra rozšiřuje princip posuvné šifry využitím násobení. Klíčem je dvojice čísel (a, b) , kde $a, b \in \mathbb{Z}_{26}$. Šifrovací funkce je definována jako:

$$e_K(x) = (ax + b) \bmod 26, \quad (4)$$

$$d_K(y) = a^{-1}(y - b) \bmod 26. \quad (5)$$

Aby bylo možné zprávu jednoznačně dešifrovat, musí být splněna podmínka, že největší společný dělitel $\gcd(a, 26) = 1$. V opačném případě by neexistovala inverzní hodnota a^{-1} nutná pro dešifrování.

Substituční šifra

Zatímco posuvná šifra pouze posouvá abecedu, obecná substituční šifra nahrazuje každý znak otevřeného textu jiným znakem podle libovolné permutace π . Klíčem je tedy samotná permutace abecedy. Ačkoliv je velikost klíčového prostoru obrovská ($26! = 4 \cdot 10^{26}$), šifra nezakrývá statistické vlastnosti jazyka a je náchylná k frekvenční analýze.¹

Vigenèrova šifra

Vigenèrova šifra je nejznámějším zástupcem polyalfabetických šifer. Na rozdíl od předchozích metod, kde se stejné písmeno vždy šifruje stejně,² zde závisí šifrování na pozici znaku. Klíčem je vektor $K = (k_1, k_2, \dots, k_m)$ délky m . Zpráva se šifruje tak, že se klíč periodicky opakuje:

$$e_K(x_1, x_2, \dots, x_m) = (x_1 + k_1, x_2 + k_2, \dots, x_m + k_m), \quad (6)$$

$$d_K(y_1, y_2, \dots, y_m) = (y_1 - k_1, y_2 - k_2, \dots, y_m - k_m). \quad (7)$$

Díky tomu jsou nejčastější znaky otevřeného textu³ šifrovány pokaždé na jiný znak, čímž se jejich vysoká četnost rozptýlí mezi více symbolů. To činí kryptoanalýzu náročnější než u jednoduché substitute.

2.2.2 Proudové šifry

Proudové šifry generují z krátkého klíče K dlouhou posloupnost pseudonáhodných bitů, označovanou jako *keystream* $z = z_1, z_2, \dots$. Šifrování otevřeného textu x probíhá operací XOR⁴ po jednotlivých bitech:

$$y_i = x_i \oplus z_i. \quad (8)$$

Dešifrování je identická operace, jelikož $(A \oplus B) \oplus B = A$. Bezpečnost proudové šifry zcela závisí na kvalitě a nepředvídatelnosti generátoru keystreamu.

¹Frekvenční analýza je metoda prolomení šifry, která využívá skutečnosti, že v přirozeném jazyce se některé znaky vyskytují častěji než jiné. Útočník může analyzovat četnosti jednotlivých znaků v šifrovaném textu a porovnat je s typickými četnostmi v daném jazyce. Tím lze odhalit pravděpodobné substitute a postupně rekonstruovat původní text i bez znalosti klíče.

²Tento přístup se označuje jako monoalfabetické šifry.

³Například písmeno E pro anglickou abecedu.

⁴Sčítání modulo 2.

LFSR (Linear Feedback Shift Register)

Jednou z nejčastějších metod pro generování keystreamu u proudových šifer je *posuvný registr s lineární zpětnou vazbou* (LFSR). LFSR délky L se skládá z L buněk (stavů) $s_0, s_1, \dots, s_{L-1}$ a lineární zpětné vazby definované koeficienty $c_0, c_1, \dots, c_{L-1}$. V každém kroku se obsah registru posune a nový bit je vypočten sečtením vybraných předchozích bitů podle koeficientů, vypadlý bit je použit jako výstup keystreamu. Tento výpočet lze formálně zapsat následovně:

$$s_{i+L} = \sum_{j=0}^{L-1} c_j s_{i+j} \bmod 2. \quad (9)$$

2.2.3 Blokové šifry

Blokové šifry zpracovávají otevřený text po pevně daných blocích. Na rozdíl od proudových šifer, které šifrují data bit po bitu, bloková šifra rozdělí vstupní zprávu na bloky a každý blok změní jako celek pomocí klíče. V mé aplikaci využívám jako zástupce blokových šifer algoritmus DES.

DES (Data Encryption Standard)

DES je symetrická bloková šifra standardizovaná v USA v roce 1977 [13]. Pracuje s bloky o délce 64 bitů a klíčem o délce 64 bitů.⁵ Celý algoritmus je postaven na architektuře tzv. *Feistelovy funkce* a opakuje se v 16 identických krocích (rundách). Vzhledem k tomu, že všechny rundy provádějí totožné operace, aplikace z důvodu přehlednosti vizualizuje pouze průběh první rundy.

Základem šifry je Feistelova funkce, která rozděljuje vstupní blok zprávy na dvě stejné poloviny: levou (L_0) a pravou (R_0). V každé z 16 rund dochází k transformaci, kde levá polovina v nové rundě přebírá hodnotu pravé poloviny z předchozí rundy. Nová pravá polovina vznikne jako výsledek operace XOR mezi starou levou polovinou a výsledkem tzv. *rundovní funkce* f :

$$L_i = R_{i-1}, \quad (10)$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i). \quad (11)$$

⁵Z tohoto klíče se však reálně využívá pouze 56 bitů, protože každý osmý bit slouží výhradně ke kontrole parity a do šifrovacího procesu nevstupuje.

Důležitou vlastností této konstrukce je symetrie mezi šifrováním a dešifrováním. Po provedení první rundy jsou výsledné poloviny před závěrečnou permutací prohozeny ($R_1 + L_1$), takže ze šifrovaného textu lze po počáteční permutaci IP přímo číst R_1 jako levou a L_1 jako pravou polovinu. Dešifrování pak probíhá následovně:

- Původní R_0 lze získat přímo, protože $L_1 = R_0$.
- Původní L_0 lze získat aplikací rundovní funkce na L_1 (tedy na R_0 , protože $L_1 = R_0$) se stejným klíčem K_1 a XORem s R_1 :

$$R_1 \oplus f(L_1, K_1) = (L_0 \oplus f(R_0, K_1)) \oplus f(R_0, K_1) = L_0. \quad (12)$$

Pro dešifrování celého 16rundového DESu tak lze použít identický algoritmus, pouze se obrátí pořadí použitých rundovních klíčů ($K_{16}, K_{15}, \dots, K_1$). Samotná rundovní funkce f zajišťuje v šifře vlastnosti konfuze⁶ a difuze.⁷ Skládá se z expanze dat (z 32 na 48 bitů), XOR s příslušným rundovním klíčem K_i , substituce pomocí tzv. *S-boxů*, které data zredukuje zpět na 32 bitů, a závěrečné permutace dle P-boxu. Rundovní klíče K_i o délce 48 bitů algoritmus v každém kroku dynamicky generuje z hlavního 64bitového klíče pomocí předem definovaných permutací a bitových posunů.

Přestože byl algoritmus DES po desetiletí hojně využíván, dnes se považuje za nespolehlivý. Hlavním důvodem je krátká délka klíče. Prostor $2^{56} \approx 7,2 \cdot 10^{16}$ možných klíčů je pro moderní výpočetní techniku příliš malý. Již v roce 1998 sestavila organizace Electronic Frontier Foundation specializovaný hardware, který dokázal klíč odhalit útokem hrubou silou za 56 hodin [13]. Standard DES byl proto postupně nahrazen bezpečnějšími algoritmy, jako jsou 3DES a moderní AES (Advanced Encryption Standard).

⁶*Konfuze* označuje vlastnost šifry, při níž každý bit šifrovaného textu závisí na co největším počtu bitů klíče.

⁷*Difuze* označuje vlastnost šifry, při níž každý bit otevřeného textu ovlivňuje co největší počet bitů šifrovaného textu.

2.3 Asymetrické šifry

Zatímco symetrická kryptografie vyžaduje, aby se odesílatel a příjemce předem dohodli na tajném sdíleném klíči, asymetrická kryptografie⁸ tento problém výměny klíčů eliminuje. Koncept byl poprvé veřejně představen Diffiem a Hellmanem v roce 1976 [3]. Hlavní myšlenkou je použití dvou odlišných, ale matematicky svázaných klíčů:

- **Veřejný klíč:** Slouží k šifrování zpráv a může být volně přístupný komukoliv.
- **Soukromý klíč:** Slouží k dešifrování a musí být držen v tajnosti pouze u majitele.⁹

Princip veřejného a soukromého klíče

Bezpečnost asymetrických systémů není založena na utajení algoritmu, ale na složitosti určitých matematických problémů. Základem jsou tzv. *jednosměrné funkce s padacími vrátky*. Jednosměrná funkce f je taková funkce, kterou je snadné vypočítat v jednom směru,¹⁰ ale výpočetně neproveditelné ji invertovat,¹¹ bez znalosti dodatečné tajné informace tzv. „padacích vrátek“.¹² V tomto případě je touto tajnou informací soukromý klíč.

Komunikace pak probíhá následovně: Pokud chce odesílatel (Bob) poslat důvěrnou zprávu příjemci (Alici), vyhledá její veřejný klíč K_{public} a zprávu zašifruje. Vzniklý zašifrovaný text nemůže dešifrovat nikdo,¹³ dokonce ani útočník s neomezeným výpočetním výkonem v rozumném čase. Jediný, kdo může zprávu obnovit, je příjemce (Alice), protože vlastní soukromý klíč $K_{private}$ „padací vrátka“, který proces inverze činí triviálním.

⁸Známa též jako kryptografie s veřejným klíčem.

⁹Příjemce zprávy.

¹⁰Pro dané x spočítat $y = f(x)$.

¹¹Pro dané y najít x , aby platilo $y = f(x)$.

¹²Z anglického *trapdoor*. Tento termín popisuje žádoucí matematickou vlastnost. Znamená to, že je velmi snadné do „pasti“ spadnout (zašifrovat zprávu), ale je nesmírně těžké se z ní dostat ven (zprávu dešifrovat), pokud neznáte tajný mechanismus, jak padací vrátka otevřít.

¹³Ani samotný odesílatel.

2.3.1 Kryptosystém RSA

Nejrozšířenějším a nejznámějším příkladem asymetrické šifry je kryptosystém RSA, pojmenovaný po svých autorech (Rivest, Shamir, Adleman), kteří jej publikovali v roce 1978 [11]. Bezpečnost RSA je založena na obtížnosti faktorizace velkých čísel, což je popsáno v sekci 2.3.2. Algoritmus lze rozdělit do tří fází: generování klíčů, šifrování a dešifrování.

Generování klíčů

Alice musí nejprve vytvořit svůj pár klíčů:

1. Zvolí dvě velká prvočísla p a q .
2. Vypočítá modul $n = p \cdot q$.
3. Vypočítá hodnotu Eulerovy funkce $\phi(n) = (p - 1)(q - 1)$.
4. Zvolí veřejný exponent b tak, aby platilo $1 < b < \phi(n)$ a $\gcd(b, \phi(n)) = 1$.
5. Vypočítá soukromý exponent a jako modulární inverzi b modulo $\phi(n)$:

$$a = b^{-1} \bmod \phi(n). \quad (13)$$

Jinými slovy, a je takové číslo, pro které platí $(b \cdot a) \bmod \phi(n) = 1$. Tato vlastnost zajišťuje, že se operace šifrování a dešifrování navzájem vyruší a umožní získat původní zprávu, jak ukážu v sekci 2.3.3.

Veřejným klíčem je dvojice (b, n) , soukromým klíčem je číslo a , které slouží k dešifrování zprávy, a znalost prvočísel p a q , která jsou držena v tajnosti.

Šifrování

Bob zašifruje otevřený text x pomocí Alicina veřejného klíče:

$$y = e_K(x) = x^b \bmod n. \quad (14)$$

Dešifrování

Alice pro získání původní zprávy použije svůj soukromý klíč a :

$$x = d_K(y) = y^a \bmod n. \quad (15)$$

2.3.2 Bezpečnost RSA

Bezpečnost algoritmu RSA není o utajení postupu šifrování, ale o matematické složitosti specifického problému zvaného *Obtížnost faktorizace velkého čísla*. Podstata tohoto problému spočívá v extrémní asymetrii výpočetní náročnosti mezi násobením a rozkladem čísel:

1. **Směr násobení (snadný):** Pro dvě velká prvočísla p a q je výpočetně triviální a velmi rychlé vypočítat jejich součin $n = p \cdot q$. Počítač tuto operaci zvládne v řádu mikrosekund i pro čísla o velikosti stovek cifer.
2. **Směr faktORIZACE (obtížný):** Je-li k dispozici pouze výsledné číslo n ,¹⁴ je výpočetně extrémně náročné najít původní p a q . Pro dostatečně velká čísla neexistuje žádný známý efektivní algoritmus, který by tento rozklad provedl v rozumném čase na klasickém počítači.

Tato vlastnost slouží jako ona „padací vrátka“. Oprávněný uživatel (Alice) zná čísla p a q od začátku, protože si je sama zvolila při generování klíče. Útočník (Oscar) však vidí pouze číslo n . Aby mohl vypočítat soukromý klíč a (modulární inverzi b modulo $\phi(n)$), musel by nejprve spočítat $\phi(n) = (p - 1) \cdot (q - 1)$. K tomu však nezbytně potřebuje znát hodnoty p a q . V praxi se dnes používají klíče o délce 2048, 3072 nebo 4096 bitů. Číslo n o délce 2048 bitů má přibližně 617 desítkových cifer. Rozklad takového čísla by i při použití nejlepších známých algoritmů a s využitím výkonu všech superpočítačů světa trval déle, než je předpokládaná doba existence vesmíru. Právě tato praktická neproveditelnost faktORIZACE garantuje bezpečnost přenášených dat.

2.3.3 Matematický princip dešifrování RSA

V této sekci ukážu, že proces šifrování a dešifrování RSA skutečně splňuje základní podmínku korektnosti kryptosystému představenou rovnicí (1) v kapitole 2.1. Skutečnost, že platí $d_K(e_K(x)) = x$, vychází z Eulerovy věty, která říká, že pro každé celé číslo x nesoudělné s n platí:

$$(x^{\phi(n)}) \bmod n = 1. \quad (16)$$

Protože a je modulární inverzí b modulo $\phi(n)$, platí $(b \cdot a) \bmod \phi(n) = 1$, a tedy $b \cdot a = k \cdot \phi(n) + 1$ pro nějaké celé číslo k . Proces dešifrování pak lze rozepsat následovně:

$$\begin{aligned} ((x^b)^a) \bmod n &= (x^{b \cdot a}) \bmod n \\ &= (x^{k \cdot \phi(n) + 1}) \bmod n \\ &= ((x^{\phi(n)})^k \cdot x^1) \bmod n \\ &= (1^k \cdot x) \bmod n \\ &= x \bmod n. \end{aligned} \quad (17)$$

Z rovnice je patrné, že díky vlastnostem modulární aritmetiky se vliv exponentů b a a vyruší a výsledkem je původní zpráva x .

¹⁴Což je veřejná část klíče.

3 Použité technologie

Aby byl uživatelům zajištěn okamžitý a snadný přístup bez nutnosti jakékoliv instalace, je výsledná aplikace navržena jako webová. Po jejím nasazení tak k ní lze přistupovat z jakéhokoliv zařízení s webovým prohlížečem a připojením k internetu. Výběr vhodných technologií je důležitým krokem při jejím návrhu. Vzhledem k požadavku na vysokou interaktivitu a vizualizaci algoritmů v reálném čase jsem při výběru technologií kladl důraz na reaktivitu uživatelského rozhraní. Tato kapitola popisuje soubor použitých technologií využitý při implementaci praktické části této bakalářské práce.

3.1 Frontendové technologie a standardy

Aplikaci jsem postavil na trojici základních webových standardů. Tyto technologie společně zajišťují jak sémantickou strukturu a vzhled, tak i potřebnou dynamiku a interaktivitu. Struktura uživatelského rozhraní je definována pomocí *HTML* [17]. V kontextu Vue.js slouží HTML jako dynamická šablona, která se překresluje na základě aktuálního stavu aplikace. Pro vizuální styl jsem použil *CSS* [2]. Pro konzistentní vzhled využívám CSS proměnné, které definují barevné schéma na jednom místě. Design je optimalizován primárně pro desktopová rozlišení s fixní šířkou obsahu, protože rozsáhlé šifrovací tabulky by na mobilních zařízeních ztrácely čitelnost. Důležitou technologií je jazyk *JavaScript* [6], který v prohlížeči oživuje statický obsah vytvořený pomocí HTML a CSS. Pracuje s *Document Object Modelem (DOM)* [4], čímž umožňuje programově měnit obsah stránky a reagovat na uživatelské události v reálném čase.

JavaScript jsem v aplikaci využil ve dvou rovinách. První je aplikační logika pro řízení interakce s uživatelem, kterou jsem implementoval pomocí frameworku Vue.js (popsaného v sekci 3.2). Druhou rovinou jsou samotné kryptografické výpočty. Všechny šifrovací a dešifrovací algoritmy jsem implementoval v čistém JavaScriptu bez externích knihoven, což mi dává plnou kontrolu nad každým krokem výpočtu a umožňuje tak vytahovat mezivýsledky složitějších algoritmů přímo do vizualizace.

3.2 Vue.js

Pro implementaci uživatelského rozhraní jsem zvolil framework *Vue.js* [16] ve verzi 3.4.0. Hlavním důvodem bylo vytvoření *Single Page Application (SPA)*, kde se při přechodu mezi jednotlivými šiframi nenačítá celá stránka znovu, ale překreslují se pouze potřebné části obsahu. Vue.js kombinuje deklarativní přístup se systémem reaktivity. Místo imperativní manipulace s DOM stačí v šabloně deklarovat, co má být zobrazeno. Například `<p>{{ text }}</p>` automaticky zobrazí aktuální hodnotu proměnné `text`. Reaktivní systém pak automaticky sleduje změny proměnných a překresluje jen dotčené části šablony.

Architektura a organizace kódu

Projekt využívá *Single-File Components (SFC)*, kde každá komponenta je v samostatném souboru `.vue` obsahujícím šablonu (HTML), styly (CSS) i logiku (JavaScript). Pro definici logiky komponent jsem zvolil *Options API* [10] kvůli jeho přehledné struktuře. Kód je rozdělen do sekcí:

- `data`: definice reaktivního stavu,
- `methods`: funkce pro interakci a výpočty,
- `computed`: automaticky přepočítávané hodnoty,
- `watch`: sledování změn proměnných.

Propojení logiky se šablonou zajišťují direktivy jako `v-if` pro podmíněné vykreslování, `v-for` pro cyklické generování prvků nebo `v-model` pro obousměrnou datovou vazbu s formulářovými prvky. O konkrétním použití těchto direktiv v mé aplikaci a dalších funkcích Vue.js se zmiňuji v sekci 4.3.2.

3.3 Vývojové prostředí a sestavení aplikace

Pro vývoj a sestavení aplikace používám prostředí *Node.js* [8] s balíčkovacím manažerem *npm* [9]. Node.js umožňuje spouštět JavaScript mimo prohlížeč, což je nutné pro kompilaci Vue komponent. Npm spravuje závislosti definované v souboru `package.json`. Aplikace používá Vue.js 3.4.0 a Vue Router 4.6.3 [15] pro navigaci mezi šiframi. Dále npm spouští příkazy nástroje *Vue CLI* [14] pro běh vývojového serveru nebo vytvoření produkční verze.

Vue CLI a sestavovací proces

Pro sestavení aplikace používám nástroj `Vue CLI service` (verze 5.0.9), který kompiluje Vue komponenty (`.vue` soubory) do HTML, JavaScriptu a CSS.

Práce s aplikací probíhá pomocí npm příkazů:

- **Lokální vývoj:** Příkaz `npm run serve` spouští vývojový server na `http://localhost:8080` s automatickou kompilací a hot-reload funkcionalitou (změny v kódu se okamžitě projeví v prohlížeči, což výrazně usnadňuje ladění vizualizací).
- **Produkční sestavení:** Příkaz `npm run build` generuje optimalizovanou verzi aplikace. Proces zahrnuje minifikaci (odstranění mezer, komentářů, zkrácení názvů proměnných) a tree-shaking (eliminace nepoužívaného kódu). Výsledné soubory jsou uloženy do složky `/docs`, která slouží jako publikační adresář pro GitHub Pages.

Nasazení

Aplikaci hostuji pomocí *GitHub Pages* [5], která poskytuje bezplatný hosting statických stránek přímo z Git repozitáře. Proces nasazení je jednoduchý: spustím `npm run build`, což vygeneruje optimalizované soubory do složky `/docs`, a následně změny odešlu do repozitáře. GitHub Pages automaticky zpřístupní aktualizovanou verzi na URL.¹⁵ GitHub Pages nabízí HTTPS bez manuální konfigurace protokolů SSL/TLS¹⁶, verzování historie nasazení díky integraci s Git a nulové provozní náklady. Protože aplikace funguje jako SPA (veškerá navigace probíhá bez obnovování stránky), je hosting statických souborů plně dostačující.

¹⁵<https://vym38224.github.io/sifrovac/>.

¹⁶Protokoly SSL (Secure Sockets Layer) a modernější TLS (Transport Layer Security) poskytují zabezpečenou komunikaci na internetu. Šifrují spojení mezi uživatelem a serverem, čímž chrání přenášená data před odposlechem a modifikací.

4 Návrh a implementace aplikace

Po volbě vhodných technologií popsanych v předchozí kapitole následoval samotný návrh a vývoj aplikace. Tato kapitola proto detailně přibližuje konkrétní kroky implementace. Nejprve rozebírám vnitřní strukturu projektu a postup, jakým jsem oddělil uživatelské rozhraní od samotné výpočetní logiky. Následně vysvětluji, jakým způsobem jsou vybrané kryptografické algoritmy převedeny z matematických definic do zdrojového kódu v jazyce JavaScript. Část textu je věnována i vizualizaci algoritmů. Na samotný závěr kapitoly jsem zařadil tvorbu vizuální identity projektu v podobě vlastního loga.

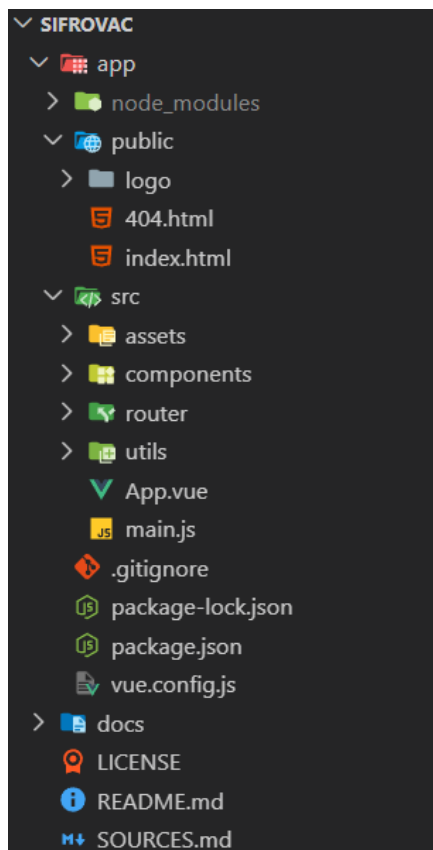
4.1 Struktura projektu

Aplikace využívá standardní strukturu Vue.js projektu (viz Obrázek 1). V kořenovém adresáři se nachází složka `/app` se zdrojovými kódy a konfigurací a složka `/docs` s produkční verzí určenou pro nasazení.

Zdrojové kódy v adresáři `app/src/` obsahují vstupní soubor `main.js` a kořenovou komponentu `App.vue`. V `main.js` vytvářím Vue instanci, registruji globální komponenty a připojuji router. Komponenta `App.vue` se pak připojí k HTML elementu s ID `app`, čímž se inicializuje celá aplikace. Projekt je dále v tomto adresáři rozdělen do složek:

- `/assets` – obrázky a CSS styly,
- `/components` – Vue komponenty (`/cipher` pro jednotlivé šifry, `/common` pro sdílené komponenty, `/layout` pro navigaci a úvodní stránku),
- `/router` – definice směrování mezi jednotlivými šiframi,
- `/utils` – implementace šifrovacích algoritmů a pomocné funkce.

V adresáři `/app/public` je vstupní soubor `index.html`, soubor `404.html` pro zpracování neexistujících URL na GitHub Pages a složka `/logo` s ikonou aplikace.



Obrázek 1: Struktura projektu aplikace

4.2 Logika šifer

Veškerá kryptografická logika aplikace je oddělena od prezentační vrstvy¹⁷ a soustředěna ve složce `/src/utils/ciphers`. Každá šifra má vlastní soubor, který exportuje šifrovací a dešifrovací funkce a další pomocné funkce. Tato architektura zajišťuje, že Vue komponenty neobsahují výpočetní logiku, ale pouze volají dané funkce a starají se o reaktivní zobrazení výsledků. Všechny algoritmy jsou implementovány v čistém JavaScriptu bez závislosti na externích knihovnách, což umožňuje detailní kontrolu nad každým krokem výpočtu a usnadňuje vizualizaci průběhu šifrování, díky využití pomocných funkcí pro zobrazení mezivýsledků a stavů algoritmu.

Implementace všech algoritmů vychází z matematických definic popsaných v kapitole 2. Základní šifry pracují pouze s velkými písmeny anglické abecedy (A–Z), mezerami a tečkami, zatímco pokročilejší algoritmy využívají pro výpočty i binární či desítkovou reprezentaci dat. K té se dostávají tak, že uživatelský vstup ve formě znaků převedou pomocí ASCII¹⁸ na odpovídající číslo, případně dále

¹⁷Prezentační vrstva označuje část aplikace zodpovědnou za zobrazení dat uživateli.

¹⁸Jedná se o standardní znakovou sadu, která přiřazuje znakům anglické abecedy, číslicím a dalším symbolům unikátní číselné hodnoty (např. písmeno Z má hodnotu 90).

na jeho binární podobu. Pro povolení mezer a teček v otevřeném textu jsou tyto znaky ošetřeny jako speciální případy vstupu, které neprocházejí běžným šifrovacím procesem, ale jsou přímo přeneseny do výsledného šifrovaného textu.

V následujícím textu postupně procházím jednotlivé kryptografické metody a popisuji způsob, jakým jsem je v jazyce JavaScript naprogramoval. U každého algoritmu pak uvádím konkrétní ukázky kódu, které neobsahují výpis šifrovacích funkcí, ale ukazují jen ta nejzajímavější implementační rozhodnutí a výzvy, které při převodu matematických vzorců do praxe nastaly.

Implementace posuvné šifry

Posuvná šifra posouvá každý znak abecedy o pevný počet pozic daný klíčem `shift`. K tomu se využívá operace modulo, která zajistí, že posun zůstane v rámci 26 písmen anglické abecedy. Dešifrování je stejné jako šifrování, pouze se pro parametr `shift` použije záporné znaménko, tedy hodnoty `shift` budou v rozsahu -25 až 0. Rozdíl indexu a klíče může nabývat záporných hodnot a jelikož operátor `%` v jazyce JavaScript nepočítá striktní matematické modulo, ale zbytek po dělení (který zachovává záporné znaménko), je v kódu před jeho aplikací přičtena hodnota 26. Tím je bezpečně zajištěno, že výsledný index bude vždy kladný a bude správně odpovídat prvkům množiny $\mathbb{Z}_{26}$.

Zdrojový kód 1: Korekce záporného modula

```
1 const newIndex = ((oldIndex + shift) + 26) % 26;
```

Tento vzorec zajistí, že výsledný index bude vždy v rozsahu 0 až 25, i když `oldIndex + shift` vyjde záporně. Stačí, aby `shift` byl mezi 0 a 25 (což aplikace kontroluje), a šifrování i dešifrování budou fungovat správně.

Implementace afinní šifry

Afinní šifra rozšiřuje Posuvnou šifru o multiplikativní složku (parametr a). Parametr a musí být nesoudělný s hodnotou 26, aby existovala jeho modulární inverze potřebná pro dešifrování. Tato podmínka je v aplikaci ověřena pomocnou funkcí `inverseA`, která prohledává hodnoty $1 \leq x < 26$ a hledá takové x , pro které platí $a \cdot x \bmod 26 = 1$. Pokud takové x existuje, vrací ho jako inverzi. Pokud ne, vrací `null`,¹⁹ což signalizuje neplatný klíč.

Zdrojový kód 2: Výpočet modulární inverze

```
1 export function inverseA(a) {
2   for (let x = 1; x < 26; x++) {
3     if ((a * x) % 26 === 1) return x;
4   }
5   return null;
6 }
```

¹⁹`null` je v JavaScriptu speciální hodnota vyjadřující záměrnou absenci hodnoty.

Pokud platná inverze neexistuje (pomocná funkce vrátí hodnotu `null`), prezentační vrstva na tuto skutečnost okamžitě reaguje. Uživateli je zobrazeno upozornění na neplatný klíč a aplikace mu znemožní spustit proces šifrování či dešifrování. Tento chybový stav je navíc přímo provázán s vizualizací algoritmu. Uživatel na obrazovce názorně vidí matematický důvod, proč je taková hodnota nepřijatelná. Aplikace ukáže, že výsledné mapování znaků abecedy přestává tvořit bijektivní funkci. Zobrazení ztrácí vlastnost prostosti, což v praxi znamená, že by se více různých písmen otevřeného textu zašifrovalo na stejný znak a text by nebylo možné jednoznačně dešifrovat zpět.

Implementace substituční šifry

Z programátorského hlediska je implementace obecné substituční šifry velmi přímočará. Klíčem je řetězec 26 znaků, který představuje libovolnou permutaci anglické abecedy. Samotný proces šifrování a dešifrování nevyžaduje složité matematické operace, ale spočívá v jednoduchém mapování znaků na základě jejich indexů. Při šifrování algoritmus nejprve zjistí zda se jedná o písmeno abecedy. Pokud ano, vyhledá pozici aktuálního znaku otevřeného textu v abecedě a nahradí jej znakem, který se nachází na totožné pozici v poskytnutém klíči.

Zdrojový kód 3: Šifrování pomocí mapování indexů

```
1 const position = ALPHABET.indexOf(char);  
2 result += permutation[position];
```

Dešifrování probíhá přesně inverzně. Pro daný zašifrovaný znak se hledá jeho pozice v permutačním klíči, čímž algoritmus získá původní znak na stejném indexu v abecedě.

Implementace Vigenèrovy šifry

Vigenèrova šifra je polyalfabetická substituční šifra, kde je každý znak otevřeného textu posunut o hodnotu odpovídajícího znaku klíče, který se cyklicky opakuje. Zatímco teoretický vzorec (6) počítá s čistým textem bez mezer a teček, reálná implementace musí řešit i tyto znaky. Kód proto využívá vlastní ukazatel `keyIndex`, který se inkrementuje pouze při zpracování písmene abecedy. Mezery a tečky tak nenarušují periodu klíče.

Zdrojový kód 4: Řízení indexu klíče při ignorování mezer a teček

```
1 if (ALPHABET.includes(char)) {
2   const keyChar = key[keyIndex % key.length];
3
4   const textIndex = ALPHABET.indexOf(char);
5   const keyCharIndex = ALPHABET.indexOf(keyChar);
6
7   const newIndex = (textIndex + keyCharIndex) % 26;
8   result += ALPHABET[newIndex];
9   keyIndex++;
10 } else if (char === '_' || char === '.') {
11   result += char;
12 }
```

Implementace proudové šifry s LFSR

Proudová šifra využívá lineární zpětnovazební posuvný registr (LFSR) pro generování pseudonáhodného proudu klíčů (keystream). Vstupem je počáteční vektor a koeficienty zpětné vazby, oba definované jako binární řetězce délky m . Generování zpětné vazby spočívá ve výpočtu operace XOR přes aktivní bity registru.²⁰

Zdrojový kód 5: Výpočet zpětné vazby LFSR

```
1 export function calculateFeedback(register, coefficients) {
2   let xorResult = 0;
3   for (let i = 0; i < register.length; i++) {
4     if (coefficients[i] === "1") {
5       xorResult ^= parseInt(register[i]);
6     }
7   }
8   return xorResult.toString();
9 }
```

V každém kroku algoritmu je výstupní bit odebrán z posledního místa registru, následně je vypočtena nová zpětná vazba a ta je vložena na jeho začátek. Výsledný proud klíčů se skládá z posloupnosti výstupních bitů. Samotné šifrování probíhá jako aplikace operace XOR mezi binární reprezentací vstupních znaků²¹ a vygenerovaným proudem klíčů. Protože dvojnásobné použití XOR se stejným klíčem vrátí původní hodnotu, je dešifrování logicky totožné se šifrováním. Z tohoto důvodu funkce decrypt v implementaci pouze volá existující funkci encrypt.

²⁰Tedy bity, jejichž koeficient má hodnotu 1.

²¹Kde každý znak odpovídá 8 bitům ASCII hodnoty.

Implementace DES

Implementace šifry DES (Data Encryption Standard) pokrývá průběh jedné rundy Feistelovy funkce se všemi operacemi. Algoritmus je navržen tak, aby vrátil dostatek mezivýsledků pro detailní vizualizaci celého výpočtu v uživatelském rozhraní. Pevné permutační a substituční tabulky (IP, IP^{-1} , E, P, PC-1, PC-2 a S-boxy) jsou izolovány ve složce `/src/utis/constants` v samostatných souborech `permutationTables.js` a `sboxes.js`.

Samotný průběh šifrování (`encrypt`) zahrnuje počáteční permutaci IP, rozdělení bloku na L0 a R0, aplikaci Feistelovy funkce na R0 s prvním rundovním klíčem K1, prohození polovin R1 + L1 a závěrečnou inverzní permutaci `IP_INV`. Pro lepší představu o tom, jak tento proces vypadá, uvádím zkrácenou ukázkou implementace první šifrovací rundy DES (viz kód 6).

Zdrojový kód 6: Průběh šifrovací rundy DES

```
1 export function encrypt(text, key) {
2   const permuted = applyPermutation(text, IP);
3
4   const L0 = permuted.slice(0, 32);
5   const R0 = permuted.slice(32, 64);
6
7   const K1 = generateFirstRoundKey(key);
8
9   const { pBoxOutput } = feistelFunction(R0, K1);
10
11  const L1 = R0;
12
13  const R1 = xorOperation(L0, pBoxOutput);
14
15  const combined = R1 + L1;
16
17  const encrypted = applyPermutation(combined, IP_INV);
18
19  return encrypted;
20 }
```

Proces dešifrování (`decrypt`) využívá symetrie Feistelovy funkce popsané v sekci 2.2.3. Implementace proto aplikuje stejný postup jako šifrování, přičemž klíč K_1 zůstává totožný. Protože aplikace vizualizuje pouze první rundu DESu, je implementováno generování pouze prvního rundovního klíče K_1 . Ten se generuje z hlavního klíče, který představuje 64 bitů. Proces zahrnuje PC-1 permutaci (redukci z 64 na 56 bitů s odstraněním paritních bitů),²² rozdělení na dvě 28bitové poloviny C a D , jejich rotaci o 1 bit doleva a finální permutaci zřetězeného výsledku pomocí tabulky PC-2 na 48 bitů.

²²Každý osmý bit 64bitového klíče slouží pouze ke kontrole parity a do šifrování se nezapočítává.

Implementace RSA

Implementace RSA staví na modulární aritmetice velkých čísel. Proces generování klíčů začíná volbou dvou prvočísel p a q . Vypočtený modul $n = p \cdot q$ musí být striktně větší než 90, aby dokázal obsáhnout číselnou reprezentaci všech využívaných znaků.²³ Následně se z Eulerovy funkce $\phi(n) = (p - 1)(q - 1)$ určí veřejný exponent b splňující podmínku nesoudělnosti $\gcd(b, \phi(n)) = 1$. Soukromý exponent a je poté vypočítán jako modulární inverze $a = b^{-1} \bmod \phi(n)$ pomocí rozšířeného Euklidova algoritmu.

Výzvou implementace bylo správné zpracování velkých číselných hodnot. Jazyk JavaScript ukládá čísla ve formátu s plovoucí desetinnou čárkou, což by při výpočtech typu $m^b \bmod n$ vedlo u velkých exponentů k nevyhnutelné ztrátě přesnosti. Tento problém řeší funkce `modPow` (viz kód 7), která veškeré operandy převádí na datový typ `BigInt`. Algoritmus navíc nepoužívá naivní násobení,²⁴ ale implementuje metodu postupného umocňování, čímž redukuje počet iterací z $O(\exp)$ na $O(\log \exp)$.

Zdrojový kód 7: Modulární umocňování s využitím `BigInt`

```
1 export function modPow(base, exp, mod) {
2   if (mod === 1) return 0;
3
4   let baseBig = BigInt(base);
5   let expBig = BigInt(exp);
6   let modBig = BigInt(mod);
7   let result = 1n;
8
9   baseBig = baseBig % modBig;
10
11  while (expBig > 0n) {
12    if (expBig % 2n === 1n) {
13      result = (result * baseBig) % modBig;
14    }
15    expBig = expBig / 2n;
16    baseBig = (baseBig * baseBig) % modBig;
17  }
18
19  return Number(result);
20 }
```

Při šifrování je číselná hodnota otevřeného textu m zakódována do podoby $c = m^b \bmod n$. Výsledný šifrový text následně tvoří posloupnost těchto čísel oddělených mezerami. Při dešifrování je vstupní řetězec rozdělen podle mezer a každá hodnota je dekodována operací $m = c^a \bmod n$.

²³Znaky jsou převáděny na čísla pomocí ASCII. Největším využívaným znakem je písmeno Z s hodnotou 90, proto musí platit $n > 90$.

²⁴Naivní násobení by znamenalo opakované násobení základu a pro výpočet m^b by se provedlo $b - 1$ násobení, což je při velkých exponentech výpočetně náročné.

4.3 Vizualizace

Hlavním přínosem aplikace je schopnost transformovat kryptografické operace do srozumitelné vizuální podoby. Toho je dosaženo díky reaktivnímu systému frameworku Vue.js, který automaticky zajišťuje synchronizaci mezi vnitřním stavem aplikace a uživatelským rozhraním. Tato kapitola se věnuje návrhu vizualizace pro jednotlivé šifry a popisuje konkrétní implementační detaily, které umožňují uživatelům sledovat průběh šifrování a dešifrování v reálném čase.

4.3.1 Návrh vizualizace

Návrh vhodného znázornění pro jednotlivé šifry představoval jednu z hlavních výzev této práce. Každý kryptografický algoritmus funguje na zcela odlišných principech, a proto nebylo možné použít jednotnou vizualizační šablonu. Pro každou metodu bylo nutné individuálně analyzovat její chování a následně navrhnout vizualizaci, která by co nejlépe odpovídala jejímu vnitřnímu fungování.

Zatímco u historických šifer se nabízí využití reálných fyzických analogií, u moderních algoritmů je situace mnohem složitější. Například u posuvné šifry se jako ideální reprezentace ukázalo interaktivní šifrovací kolo, které studentům přirozeně demonstruje mechanický posun abecedy. U Afinní nebo Vigenèrovy šifry zase vizualizace využívá dynamické mapovací tabulky, jež jasně ukazují vztahy mezi otevřeným a šifrovým textem.

Naopak u pokročilejších metod, jako je bloková šifra DES nebo asymetrický kryptosystém RSA, nelze matematické operace snadno převést do intuitivních fyzických metafor. Vizualizace těchto algoritmů proto spíše připomíná detailní průchod samotným výpočtem. V těchto případech rozhraní uživatele krok za krokem provází transformací dat, čímž odkrývá jinak skryté vnitřní stavy šifry.

4.3.2 Implementace vizualizace pomocí Vue.js

Základem interaktivní vizualizace je definice vnitřního stavu komponenty v rámci sekce `data`. Zde jsou inicializovány veškeré reaktivní proměnné, jako je vstupní text, parametry klíče nebo další proměnné nezbytné pro vykreslování. Tyto proměnné tvoří živé propojení mezi logikou a HTML šablonou, díky čemuž vizualizační prvky nejsou definovány staticky, ale plně se odvíjejí od aktuálního stavu aplikace.

Sběr dat od uživatele a reakci na jeho podněty zajišťují direktivy pro interakci. Obousměrná datová vazba realizovaná pomocí `v-model` okamžitě promítá změny ze vstupních polí do reaktivního stavu. Další uživatelské události, jako je kliknutí na tlačítko nebo interakce s ovládacími prvky, aplikace zachycuje pomocí direktivy `v-on` (zkráceně `@`). Na tyto změny v datech rozhraní následně okamžitě reaguje úpravou samotné vizualizace. Praktickou ukázkou tohoto propojení lze vidět u nastavení klíče posuvné šifry pomocí posuvníku (viz kód 8), kde se kombinují direktivy pro vazbu dat i reakci na vstup.

Zdrojový kód 8: Ukázka použití direktiv u posuvníku klíče

```
1 <input
2   id="shift-slider"
3   type="range"
4   min="0"
5   max="25"
6   step="1"
7   v-model.number="shift"
8   @input="rotateWheel"
9 />
```

Direktiva `v-model.number` zajišťuje, že hodnota posuvníku je automaticky převedena na číslo a uložena do reaktivní proměnné `shift`. Direktiva `@input` pak volá funkci `rotateWheel`, která aktualizuje vizualizaci posuvné šifry v reálném čase podle aktuální hodnoty klíče (a to tak, že otočí obrázek šifrovacího kola o daný úhel).

Pro šifry, kde vizualizaci algoritmu dynamicky sestavuji z HTML elementů, používám další direktivy: `v-if` pro podmíněné vykreslování částí šifrovacího procesu a `v-for` pro generování opakujících se prvků, například řádků substitučních tabulek. Důležitým doplňkem je direktiva `v-bind` (zkráceně `:`), která umožňuje dynamicky nastavovat libovolné atributy HTML elementů na základě aktuálních dat. V praxi to znamená, že lze nejen měnit CSS třídy a styly, ale také například hodnoty různých atributů podle stavu aplikace. To je v kontextu vizualizace velmi užitečné pro zvýraznění aktuálně zpracovávaných znaků, chybových stavů nebo jiných relevantních informací. Díky tomuto přístupu framework na pozadí automaticky manipuluje s DOM a aktualizuje v prohlížeči pouze ty části stránky, ve kterých skutečně došlo k datové změně.

4.4 Logo

Logo jsem vytvořil v programu Affinity[1], jako vektorovou grafiku. Myšlenka vznikla skicováním na papíře, kdy jsem zkoušel kombinace prvního písmene názvu aplikace *Š* s různými objekty z oblasti kryptografie. Nejvíce se osvědčil návrh spojující písmeno *Š* s klíčem, jelikož klíč je přirozenou vizuální metaforou kryptografie. Spodní část písmene tvoří ozubený konec klíče a horní část jeho hlavičku. Výsledné logo je zobrazeno na Obrázku 2. Při tvorbě jsem vycházel z fontu MS UI Gothic, jehož geometrický tvar písmene *S* jsem dále upravil do výsledné podoby loga.



Obrázek 2: Logo aplikace

5 Presentace dosažených výsledků

V této kapitole představuji výslednou podobu webové aplikace a demonstruji její funkčnost na vybraných kryptografických algoritmech. Hlavním cílem je ukázat, jak navržené uživatelské rozhraní a interaktivní prvky napomáhají k pochopení jednotlivých šifrovacích procesů. Závěr kapitoly je věnován kompatibilitě mezi prohlížeči a dalšího rozšíření aplikace o další šifry.

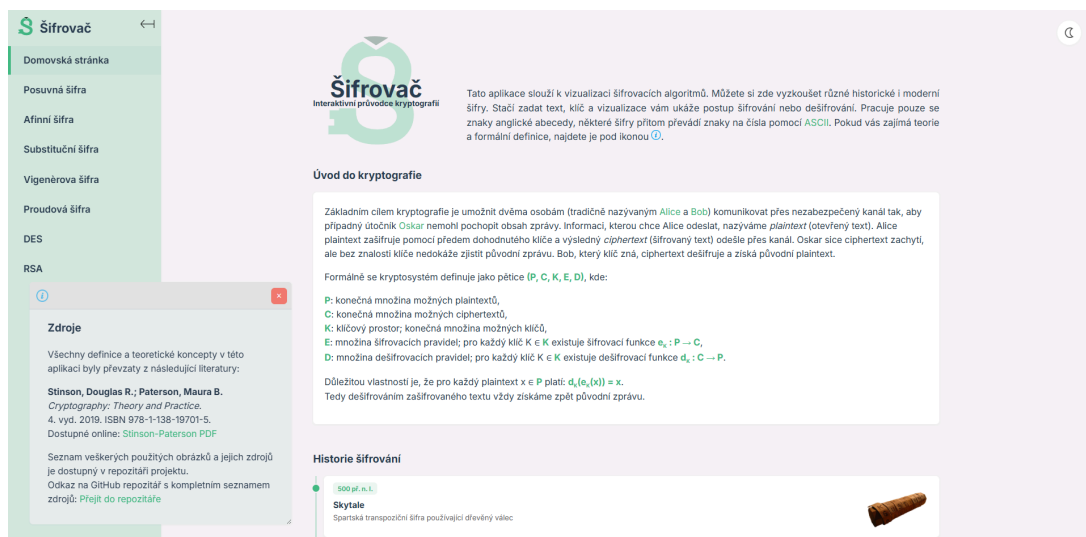
5.1 Ukázky průchodu aplikací

Pro demonstraci možností nástroje jsem zvolil čtyři typické scénáře. Prvním scénářem je domovská stránka, kterou uživatel vidí při prvním otevření aplikace. Zbývající tři scénáře pokrývají odlišné kategorie šifer i odlišné přístupy k samotné vizualizaci: posuvnou šifru, proudovou šifru LFSR a asymetrický kryptosystém RSA. Všechny implementované šifry sdílejí jednotný uživatelský koncept, který studenty vede od intuitivního zkoumání až k formální matematice. Tento proces se skládá ze tří kroků, které se napříč scénáři opakují:

1. **Vizualizace a experimentování:** Uživatel nastaví parametry šifry, zadá text a sleduje proces šifrování.
2. **Ověření správnosti:** Uživatel může zkopírovat výsledný šifrový text, vložit jej zpět na vstup a spustit dešifrování, čímž si v praxi ověří správnost procesu.
3. **Propojení s teorií:** Jakmile si student vytvoří intuitivní představu o fungování šifry, může si pomocí informačního tlačítka zobrazit teoretický základ a formální matematickou definici, která je díky předchozí vizuální zkušenosti mnohem srozumitelnější.

Scénář 1: Domovská stránka aplikace

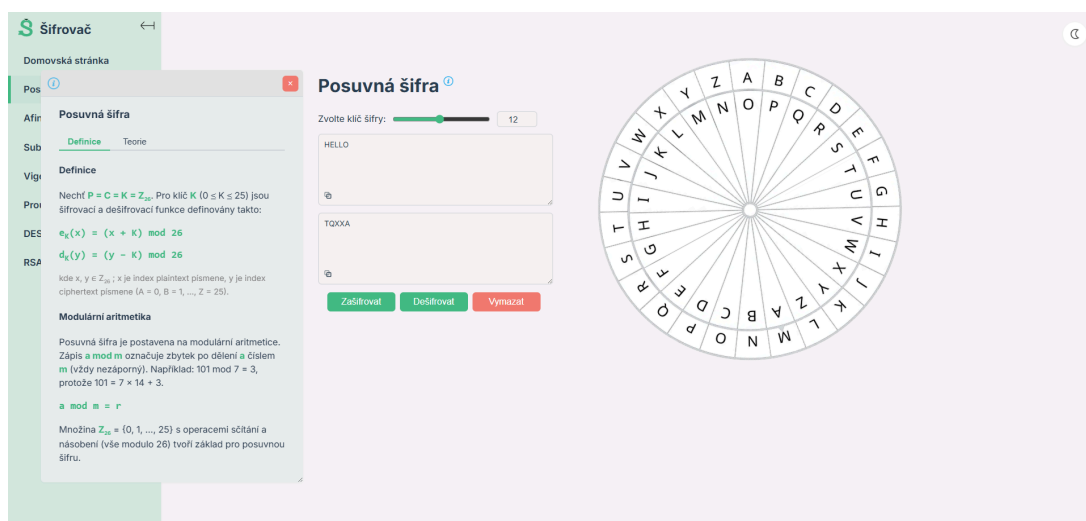
Než přistoupíme ke konkrétním šifrám, je nutné představit domovskou stránku, která se uživateli po spuštění aplikace zobrazí jako první (viz Obrázek 3). Její primární rolí je uvést studenta do problematiky a vysvětlit mu, jak s nástrojem pracovat. Po kliknutí na informační tlačítko se zobrazí seznam použité literatury a zdroje obrázků. Hlavní část stránky tvoří úvod do kryptografie, který slouží jako teoretický základ a motivace pro další zkoumání. Pokud uživatel zascrolluje níže (což již není na obrázku zachyceno), nalezne stručný přehled historie a slovníček základních pojmů. Právě na tyto pojmy se následně odkazuje v teoretických částech u jednotlivých šifer. Domovská stránka tak funguje jako ideální vstupní bod, jenž uživateli poskytuje nezbytný kontext před samotnou interakcí s algoritmy, které si může libovolně vybírat z přehledného levého menu.



Obrázek 3: Domovská stránka aplikace

Scénář 2: Posuvná šifra

První ukázka (viz Obrázek 4) představuje práci s jednoduchou posuvnou šifrou. Jakmile uživatel zvolí klíč pomocí posuvníku, tato akce okamžitě spustí rotaci šifrovačského kola, které vizuálně znázorňuje posun abecedy. Po zadání otevřeného textu a kliknutí na tlačítko „Zašifrovat“ se vygeneruje výsledek, přičemž uživatel ve vizualizaci přímo vidí aktuální mapování mezi zdrojovými a cílovými znaky. Možnost volně experimentovat s posuvníkem a okamžitě pozorovat změny v mapování pomáhá studentům pochopit základní princip modulární aritmetiky a její význam pro zachování struktury abecedy.



Obrázek 4: Rozhraní posuvné šifry

Scénář 3: Proudová šifra s LFSR

Druhý scénář (viz Obrázek 5) se zaměřuje na proudovou šifru s LFSR. Uživatel nastaví velikost registru, počáteční vektor a koeficienty zpětné vazby. Vizualizace vstupní text následně automaticky převádí do binární podoby (pokud už není v binární podobě). Během šifrování si uživatel musí sám generovat proud klíčů a krok za krokem sledovat, jak se mění stav registru a jak se generuje samotný proud klíčů. Poslední část vizualizace demonstruje samotný proces šifrování, kdy se jednotlivé bity otevřeného textu spojují pomocí operace XOR s bity vygenerovaného proudu klíčů. Díky tomuto detailnímu pohledu získá student představu o fungování posuvných registrů v praxi, což mu následně usnadní pochopení matematického zápisu algoritmu. Zároveň pochopí operaci XOR a její důležitou vlastnost symetrie, která umožňuje, že stejný proces lze použít pro šifrování i dešifrování.

Šifrovač

Domovská stránka

Pos

Afir

Sub

Vigi

Pro

DES

RSA

Proudová šifra

Registr m: 5 Počáteční vektor: 10001

Koeficienty: 11001

Text Bity

HELLO

1100001101010001010000000000100111110

Zašifrovat Dešifrovat Vymazat

Převod znaku na bity (ASCII)

H E L L O

01001000 01000101 01001100 01001100 01001111

Generování proudu klíčů

0 0 1 1 0

Zpětná vazba: 0 XOR 0 XOR 0 = 0

Generovat vč

Proud klíčů:

1000101100010110001011000101100010110001

Šifrování vstupu pomocí operace XOR

Vstup: 0100100001000101010011000100110001001111

P. klíč: 10001011000101100010110001011000101100010110001

Výsledek: 110000110101001101100000000101001111110

Obrázek 5: Rozhraní proudové šifry s LFSR

Scénář 4: Kryptosystém RSA

Závěrečný scénář (viz Obrázek 6) se věnuje asymetrickému kryptosystému RSA. Zde vizualizace nestaví na fyzické metafoře, ale na detailním průchodu matematickými operacemi. Na samém začátku uživatel zadá vstupní parametry v podobě dvou prvočísel p a q a otevřený text. Následná vizualizace je rozdělena do dvou logických částí. První část se věnuje procesu výpočtu klíče, který začíná výpočtem modulu n a Eulerovy funkce $\phi(n)$. Poté si uživatel zvolí z nabízených platných možností veřejný exponent b a soukromý exponent a . Druhá část se zaměřuje na samotné šifrování či dešifrování, kde uživatel pozoruje převod textu na číselnou reprezentaci, umocňování a aplikaci modula. Tento krokovaný matematický průchod umožňuje studentovi pochopit vnitřní logiku RSA mnohem efektivněji než pouhé čtení vzorců a demonstruje mu rozdíl mezi symetrickou a asymetrickou kryptografií.

The screenshot displays the RSA encryption interface, which is divided into several sections:

- Domovská stránka** (Home page) with a navigation menu on the left.
- Pos** (Position) section with a sub-menu for **RSA**.
- Definice** (Definition) section, which includes the RSA definition and the generation of parameters.
- Generování parametrů RSA** (RSA parameter generation) section, which lists the steps for generating the key pair.
- Volba b** (Choice of b) section, which allows the user to select a public exponent b .
- RSA** section, which contains the main interface for entering the prime numbers p and q , the plaintext, and the ciphertext.
- Výpočet klíče** (Key calculation) section, which shows the steps for calculating the modulus n , Euler's totient function $\phi(n)$, and the public and private exponents b and a .

The interface is designed to be user-friendly, with clear labels and a logical flow of information. The RSA section includes input fields for the prime numbers p and q , the plaintext, and the ciphertext. The key calculation section shows the steps for calculating the modulus n , Euler's totient function $\phi(n)$, and the public and private exponents b and a .

Obrázek 6: Rozhraní pro kryptosystém RSA

5.2 Požadavky na prohlížeč a zařízení

Aplikace byla vyvíjena a testována v prohlížeči Google Chrome. Důvodem je skutečnost, že většina současných desktopových prohlížečů (např. Microsoft Edge, Opera, nebo Brave) používá stejné vykreslovací jádro Chromium, a proto je jejich chování z hlediska HTML a CSS ve většině případů velmi podobné. Kompatibilitu jsem také ověřil v prohlížeči Mozilla Firefox (jádro Gecko), kde aplikace běží bez problémů. Pro praktické použití proto doporučuji zejména prohlížeče Google Chrome a Mozilla Firefox.

Dále není optimalizována pro mobilní zařízení. Vizualizace algoritmů vyžaduje dostatečný prostor na obrazovce, aby uživatelé mohli jednotlivé kroky šifrování sledovat a porozumět jim. Na malých displejích telefonů by vizualizace ztrácela čitelnost a přehlednost, což by vedlo k ztížené orientaci v rozhraní a snížilo by to edukativní hodnotu aplikace. Z těchto důvodů doporučuji používat aplikaci primárně na desktopových počítačích.

5.3 Rozšíření o další šifry

Aplikace je navržena s ohledem na budoucí rozšiřitelnost. Díky architektuře, kde je logika šifer oddělena od prezentační vrstvy, lze relativně snadno přidat další kryptografické algoritmy. Stačí vytvořit nový soubor s implementací šifrovací a dešifrovací funkce a následně vytvořit odpovídající Vue komponentu pro vytvoření uživatelského rozhraní s vizualizací. Tímto způsobem může být aplikace postupně rozšiřována o další šifry, protokoly a kryptografické metody. Možná rozšíření mohou být například:

- **Hillova šifra:** První matematická bloková šifra, která by demonstrovala základní principy blokové šifry. Vizualizace by vycházela z lineární algebry a interaktivně by demonstrovala násobení vektorů otevřeného textu s maticí klíče.
- **Diffie-Hellmanova výměna klíčů:** Tento protokol umožňuje dvěma stranám bezpečně si vyměnit tajný klíč přes nezabezpečený kanál. Vizualizace by mohla demonstrovat proces výměny klíčů tak, jak bývá typicky znázorňován pomocí míchání barev.
- **Různé transpozice:** Například šifra Rail Fence, která využívá záměnu pozic znaků. Vizualizace by mohla ukázat, jak se znaky rozkládají do pilovitého vzoru a následně se čtou v určitém pořadí.

Závěr

Hlavním cílem této bakalářské práce bylo vytvořit interaktivní webovou aplikaci, která studentům usnadní pochopení základních i pokročilých kryptografických algoritmů. Kryptografie je obor náročný na abstraktní a matematické myšlení, a právě interaktivní vizualizace představuje velmi efektivní způsob, jak tuto počáteční bariéru při studiu překonat.

V teoretické části jsem shrnul nezbytné matematické základy a principy vybraných šifer, od jednoduché historické substituce až po moderní asymetrický kryptosystém RSA. Tyto poznatky jsem následně využil při samotném vývoji. Aplikaci jsem postavil na frameworku Vue.js, jehož reaktivní systém se ukázal jako ideální nástroj pro plynulé a dynamické propojování uživatelských vstupů s vizualizací v reálném čase. Veškerou kryptografickou logiku jsem navíc naprogramoval v čistém jazyce JavaScript, což mi poskytlo plnou kontrolu nad výpočty a umožnilo detailní krokování složitějších algoritmů.

Výsledkem mé práce je funkční a veřejně dostupná webová aplikace. Jak bylo demonstrováno na konkrétních scénářích, nástroj uživatele nejen vizuálně provází procesem šifrování, ale poskytuje mu také prostředí pro experimentování s parametry a klíči. Navržené uživatelské rozhraní tak úspěšně propojuje vizuální intuici s formální matematickou definicí a pomáhá studentům pochopit algoritmy v širších souvislostech.

Aplikace je navíc navržena s ohledem na budoucí rozšiřitelnost, což umožňuje přidávat další kryptografické metody bez nutnosti úprav stávajícího kódu. Tímto způsobem může být nástroj postupně rozšiřován.

Conclusions

The main goal of this bachelor thesis was to create an interactive web application that facilitates students' understanding of both basic and advanced cryptographic algorithms. Cryptography is a field that demands abstract and mathematical thinking, and interactive visualization represents a highly effective way to overcome this initial learning barrier.

In the theoretical part, I summarized the necessary mathematical foundations and principles of selected ciphers, ranging from simple historical substitution to the modern asymmetric RSA cryptosystem. I subsequently applied this knowledge during the actual development. I built the application on the Vue.js framework, whose reactive system proved to be an ideal tool for seamlessly and dynamically connecting user inputs with real-time visualization. Furthermore, I programmed all the cryptographic logic in pure JavaScript, which provided me with full control over the calculations and enabled detailed step-by-step execution of more complex algorithms.

The result of my work is a functional and publicly available web application. As demonstrated in the specific scenarios, the tool not only visually guides the user through the encryption process but also provides an environment for experimenting with parameters and keys. The designed user interface thus successfully connects visual intuition with formal mathematical definitions, helping students understand the algorithms in a broader context.

Additionally, the application is designed with future extensibility in mind, allowing the addition of other cryptographic methods without the need to modify the existing codebase. In this way, the tool can be gradually expanded over time.

A Obsah elektronické přílohy

Elektronická data odevzdaná v systému KI PřF UP v Olomouci obsahují veškeré soubory k textu práce, včetně zdrojových kódů potřebných ke zprovoznění aplikace. Struktura elektronické přílohy je následující:

README.txt

Textový soubor obsahující postup ke zprovoznění aplikace.

app/

Adresář se zdrojovými kódy potřebnými ke zprovoznění aplikace.

text/

Adresář obsahuje text práce ve formátu PDF, který byl vytvořený použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce. Zahrnuje také všechny (textové) přílohy a soubory potřebné pro bezproblémové vytvoření PDF dokumentu (v ZIP archivu), tj. zdrojový kód textu a vložené obrázky.

Literatura

- [1] *Affinity*. [online]. [cit. 2026-3-6]. Dostupný z: <https://www.affinity.studio/>.
- [2] *CSS – Cascading Style Sheets*. [online]. [cit. 2026-2-3]. Dostupný z: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [3] Diffie, Whitfield; Hellman, Martin E. New directions in cryptography. *IEEE Transactions on Information Theory*. 1976, roč. 22, č. 6, s. 644–654.
- [4] *DOM – Document Object Model*. [online]. [cit. 2026-2-3]. Dostupný z: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.
- [5] *GitHub Pages*. [online]. [cit. 2026-2-3]. Dostupný z: <https://pages.github.com/>.
- [6] *JavaScript*. [online]. [cit. 2026-2-3]. Dostupný z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [7] Kerckhoffs, Auguste. La cryptographie militaire. *Journal des sciences militaires*. 1883, roč. 9, s. 5–38.
- [8] *Node.js*. [online]. [cit. 2026-2-3]. Dostupný z: <https://nodejs.org/>.
- [9] *npm – Node Package Manager*. [online]. [cit. 2026-2-3]. Dostupný z: <https://www.npmjs.com/>.
- [10] *Options API*. [online]. [cit. 2026-2-3]. Dostupný z: <https://vuejs.org/api/options-state.html>.
- [11] Rivest, Ronald L.; Shamir, Adi; Adleman, Leonard. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*. 1978, roč. 21, č. 2, s. 120–126.
- [12] Singh, Simon. *Kniha kódů a šifer: tajná komunikace od starého Egypta po kvantovou kryptografii*. Druhé vyd. Praha: Dokořán, 2009. ISBN 978-80-7363-268-7.
- [13] Stinson, Douglas R.; Paterson, Maura B. *Cryptography: Theory and Practice*. 4. vyd. 2019. ISBN 978-1-138-19701-5.
- [14] *Vue CLI: Standard Tooling for Vue.js Development*. [online]. [cit. 2026-2-3]. Dostupný z: <https://cli.vuejs.org/>.
- [15] *Vue Router: The official router for Vue.js*. [online]. [cit. 2026-2-3]. Dostupný z: <https://router.vuejs.org/>.
- [16] *Vue.js*. [online]. [cit. 2026-2-3]. Dostupný z: <https://vuejs.org/>.
- [17] WHATWG. *HTML – HyperText Markup Language* [online]. [cit. 2026-2-3]. Dostupný z: <https://html.spec.whatwg.org/>.